

Ivy Homes — Property Explorer

Product Requirements Document (PRD) & Implementation Context for Antigravity

Purpose: This document gives Antigravity the complete product and engineering context before implementation. The assignment is not simply a real-estate UI task: it combines API investigation, data analysis, a working frontend, documentation-discrepancy reporting, and production deployment.

1. Assignment Context

Ivy Homes provides a property API for a city-specific internship assignment. The supplied API documentation was generated from old/internal material and is intentionally unreliable in places. The assignment states that the running API is the source of truth. Some documented endpoints may not exist, paths may differ, fields/units may differ, and some parameters may be ignored. The API itself is described as healthy and observable.

The candidate must build a working web frontend on top of the real API and produce a reproducible list of documentation discrepancies. The frontend has six required capabilities: login, listings browsing/filtering, listing detail, saved listings, rentals/projects, and an insights screen.

2. Product Vision

Product: Ivy Homes Property Explorer

Vision: A clean, reliable property discovery application that uses the real Ivy Homes API, accurately represents prices and areas, supports user favourites, and exposes useful market/data-quality insights discovered during API investigation.

Primary principle: Correctness over feature count. A small application where every number and behavior is correct is preferable to a larger application containing assumptions or inaccurate data.

3. Goals

Goal	Description
Real API integration	Use the real authentication and property endpoints; do not replace the API with mock data.
Reliable property discovery	Allow users to browse, filter, paginate, sort and inspect listings.
Persistent favourites	Users can save/remove listings and see their saved listings after refresh and re-login.
Rentals & projects	Provide usable browsing/detail experiences with correct price and area units.
Data investigation	Download the complete dataset and calculate the ten assignment answers.
Documentation audit	Reproduce and report discrepancies between documentation and actual API behavior.
Insights	Turn API analytics and discovered data-quality patterns into a useful human-facing dashboard.
Production deployment	Deploy the application to Vercel with secrets protected server-side.

4. Scope — Mandatory Frontend Features

Authentication: Real login flow; session survives page refresh; logout; clear loading/error states.

Listings: Paginated or infinite list. Filters for locality, bedrooms, price range and furnishing must actually filter even if the server does not.

Listing detail: A URL-addressable page for each listing with useful property information.

Saved listings: Add, remove and list favourites per user; persistence after reload and re-login.

Rentals & projects: Browsable rental and project experiences with correct prices and areas.

Insights: Documented analytics plus useful statistics/data-quality discoveries from the investigation.

5. Data Investigation Requirements

Before building the UI, investigate the running API and download the complete dataset. The assignment recommends pulling the whole dataset early. The API documentation says collection endpoints use page/limit pagination and return total/page/page_size/results, but this behavior must be verified against the running service.

#	Required answer	Definition
1	total_listing_records	Number of listing records retrievable from /v1/listings.
2	unique_properties	Distinct physical properties represented by those records; repeated records count once.
3	active_listings	Retrievable listings whose actual is_live value is true.
4	corrupt_listing_ids	Small set of listing records describing something impossible; sorted IDs.
5	total_monthly_rent	Sum of monthly rent for rentals in assigned locality (Thane West).
6	avg_price_per_sqft_2bhk	Mean of price/carpet_area for live 2BHK listings, excluding answers to Q4 and Q9; 2 decimals.
7	costliest_project	Project with highest maximum price: project_id + price_max_inr.
8	listings_last_7_days	Listings posted in [2026-09-03 00:00 IST, 2026-09-10 00:00 IST).
9	fake_listing_ids	Listings that are not genuine and appear intended to generate enquiries; sorted IDs.
10	projects_with_wrong_listing_count	Count of projects whose reported listing count differs from actual listing records.

6. API Integration Context

The supplied API reference identifies the base URL as **https://solve.ivy.homes**, but explicitly warns that it was not verified against the running service. Therefore Antigravity must implement a reusable API client and verify actual behavior before relying on documentation.

- **API key:** city-scoped and supplied individually. Keep it server-side; never hardcode it in client code.
- **Authentication:** POST /auth/login with email/password returns a Bearer token. Tokens are documented as valid for 24 hours; there is no refresh flow.
- **Demo users:** three demo accounts exist and share the password issued with the candidate's key.
- **Listings:** /v1/listings; detail and similar-listing endpoints are documented but must be verified.
- **Rentals:** /v1/rentals and detail endpoint.
- **Projects:** /v1/projects and detail endpoint.
- **Favourites:** GET/POST/DELETE operations.
- **Analytics:** /v1/analytics/summary.
- **Errors:** 400, 401, 403, 404, 422, 429 are documented; read useful error bodies.
- **Health:** /health is unauthenticated.

7. Recommended Technical Architecture

Use a Next.js + TypeScript web application with Tailwind CSS. Use a server-side API proxy/route-handler layer so the Ivy API key never reaches the browser. Keep the data-investigation scripts separate from the runtime frontend.

Browser → Next.js UI → Next.js server/API routes → Ivy Homes API

Suggested project structure

```
app/ login/ listings/ listings/[id]/ rentals/ projects/ projects/[id]/ favourites/ insights/ api/
components/ lib/ scripts/ data/ submission.json README.md .env.example
```

8. Core Data Flow

Stage	Action
1. Discover	Call health/auth and test documented endpoints, response shapes, pagination, filters and sorting.
2. Collect	Page through listings, rentals and projects until the complete dataset is retrieved.
3. Analyze	Run deterministic calculations and investigate non-trivial data-quality patterns.
4. Audit docs	Compare documented claims against observed API behavior and reproduce discrepancies.
5. Build API layer	Expose safe server-side routes to the frontend.
6. Build UI	Implement required pages and workflows using real data.
7. Validate	Test auth, filters, favourites, units, errors, refresh/re-login and production build.
8. Deploy	Deploy to Vercel, configure environment variables, test production.

9. UI / UX Requirements

- **Login:** simple, professional, clear errors and loading state.
- **Listings:** strong filter bar, result count, property cards, pagination, favourite control, loading skeleton and empty state.

- **Detail:** price, BHK, bathrooms, carpet/super-built-up area, floor, furnishing, facing, parking, location, project, description, verification and similar listings where available.
- **Favourites:** clear saved state and useful empty state.
- **Rentals:** make monthly rent, deposit, maintenance and area visually unambiguous.
- **Projects:** developer, status, units, towers, floors, launch/possession, RERA, area range, price range and amenities where available.
- **Insights:** summary cards, locality/BHK statistics and clearly labeled data-quality findings. Do not invent statistics.
- **Responsive:** desktop-first but usable on smaller screens. Accessibility and clarity are more important than animations.

10. Data Quality & Documentation Findings

Findings must be evidence-based. Do not report a discrepancy simply because a value looks unusual. For record-level findings, include identifiers (listing_id, project_id or phone number) where required. Use only the assignment's allowed categories:

auth · pagination · units · filters · sorting · timestamps · duplicates · completeness · data_quality · fraud · consistency · missing_endpoint · undocumented_endpoint

For each finding use: endpoint, category, documented, actual, how_found, impact, evidence. Use {id} for path parameters. Precision matters as much as recall; a short reproducible list is better than guesses.

11. Security Requirements

- Store the Ivy API key in server-side environment variables.
- Never expose the API key through client-side JavaScript, NEXT_PUBLIC variables, logs, screenshots or source control.
- Do not commit .env.local.
- Use .env.example with placeholders only.
- Do not scan other users' keys or attack the API.
- Use only the candidate's assigned API key and normal request volume.

12. Testing Acceptance Criteria

Area	Must pass
Auth	Real login, wrong credentials, refresh persistence, logout, re-login.
Listings	Pagination, locality/BHK/price/furnishing filters, sorting, detail route.
Favourites	Add/remove/list; persists after refresh and re-login; isolated per user.
Rentals	Pagination, filters, correct monthly rent and area display.
Projects	List/detail, correct price and area display.
Insights	Numbers match the completed analysis; no invented values.
Resilience	Useful handling for 401/404/422/429, network failures and empty states.
Production	Typecheck, lint, production build and deployed Vercel app work.

13. Submission Artifacts

- **Public GitHub repository** containing the frontend.
- **submission.json** at repository root using the exact required answer keys and findings structure.
- **README.md** covering setup, investigation methodology, what was distrusted, what was verified as fine, findings, and what would be done with two additional days.
- **Vercel deployment** with production environment variables configured safely.

14. Recommended Git Commit Strategy

Commit incrementally because the assignment says commit history is reviewed. Suggested commits:

```
init: scaffold Next.js application feat: add Ivy API client feat: implement authentication feat: add
API discovery scripts feat: collect complete property dataset feat: implement data analysis feat: add
documentation discrepancy findings feat: implement listings feat: implement listing details feat:
implement favourites feat: implement rentals feat: implement projects feat: add insights dashboard
test: validate API and frontend flows docs: add investigation and submission documentation chore:
prepare Vercel deployment
```

15. Implementation Order for Antigravity

Do not build the whole application in one step. Work in these phases and verify each phase before moving to the next:

- Phase 1:** Read assignment + inspect constraints; create architecture.
- Phase 2:** Scaffold Next.js project and environment configuration.
- Phase 3:** Investigate the running API; never blindly trust API_REFERENCE.md.
- Phase 4:** Download complete listings, rentals and projects.
- Phase 5:** Solve the ten questions with deterministic scripts.
- Phase 6:** Reproduce documentation discrepancies and create findings.
- Phase 7:** Build secure server-side API layer.
- Phase 8:** Build login + listings + detail.
- Phase 9:** Build favourites + rentals + projects.
- Phase 10:** Build insights using verified data.
- Phase 11:** QA, README, submission.json and incremental commits.
- Phase 12:** Deploy to Vercel and test production.

16. Explicit Non-Goals

- Do not build an unrelated AI chatbot.
- Do not replace the Ivy API with mock data.
- Do not over-engineer with unnecessary backend infrastructure.
- Do not spend the majority of the schedule on animations or visual effects.
- Do not invent data-quality findings without reproducible evidence.
- Do not expose credentials or API keys.

17. Definition of Done

The project is done when: all six frontend capabilities work against the real API; the complete dataset has been investigated; all ten answers are calculated and represented in submission.json; documentation discrepancies are reproducible and documented with evidence; README explains both successful and unsuccessful hypotheses; the application passes the final QA checklist; secrets are protected; Git history shows meaningful progress; and the production deployment works on Vercel.

18. Important Source Constraints

This PRD is based on the supplied Ivy Homes assignment statement, API reference and submission template. The assignment itself says the running API is the source of truth, so this PRD intentionally does not assert undocumented API behavior as fact. The candidate must verify actual behavior during the investigation phase.